ICS 202 – Data Structures and Algorithms

Topic 13: Graphs

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Part A: Multiple choice

Conventions: a directed edge is written `u->v`, an undirected edge `u-v`, and a weight follows in parentheses, e.g. `A->B (4)`. Unless a question says otherwise, traversals use the general traversal algorithm (after the start vertex's traversal ends, the next unvisited vertex in label order starts a new traversal), and whenever there is a choice, vertices are taken in alphabetical / numerical order of their labels.

Q1.

Old Exam Graphs

The degree of a vertex of an undirected graph is the number of edges incident to the vertex. The sum of all the degrees is equal to:

(A) twice the number of edges

(B) twice the number of vertices

(C) the number of edges

(D) the number of vertices

Answer: (A). Every edge `(u,v)` adds 1 to the degree of `u` and 1 to the degree of `v`, so the sum is $2|E|$.

My answer

Q2.

Activity 13-01 (T252)

A directed graph has $V = \{A, B, C, E, F\}$ and edges `B->A, B->E, B->F, A->E, A->F, A->C, E->C, E->F, F->E, F->C`. What are the in-degree and the out-degree of vertex `E`?

(A) in-degree 3, out-degree 2

(B) in-degree 2, out-degree 3

(C) in-degree 3, out-degree 1

(D) in-degree 5, out-degree 0

Answer: (A). Edges into E: B-¿E, A-¿E, F-¿E (3). Edges leaving E: E-¿C, E-¿F (2).

My answer

Q3.

Activity 13-01 (T252)

Consider the directed graph with vertices 1, 2, 3, 4 and edges `1->2, 1->3, 2->3, 2->4, 4->3`. Which statement is correct?

(A) The graph is strongly connected.

(B) The graph is weakly connected but not strongly connected.

(C) The graph is disconnected (its underlying undirected graph is not connected).

(D) The graph contains the cycle 1, 2, 4, 1.

Answer: (B). Ignoring directions the graph is connected, but vertex 3 has no outgoing edge, so no other vertex is reachable from 3. The graph has no cycle.

My answer

Q4.

Activity 13-01 (T252)

An undirected graph has 8 vertices: a square `a-b, b-c, c-d, d-a`, and one extra vertex hanging from each corner: `a-p, b-q, c-r, d-s`. How many different spanning trees does this graph have?

(A) 1

(B) 4

(C) 8

(D) 16

Answer: (B). The four pendant edges must be in every spanning tree; the only choice is which one of the 4 square edges to remove to break the single cycle: 4 spanning trees.

My answer

Q5.

Activity 13-01 (T252)

A directed graph has vertices A, B, C, D, E and edges `A->B, B->C, B->D, C->E, E->D, D->A`. In its adjacency matrix (rows and columns in the order A, B, C, D, E), the row of vertex `B` is:

(A) 0 0 1 1 0

(B) 1 0 1 1 0

(C) 0 1 1 1 0

(D) 0 0 1 0 0

Answer: (A). Row B has a 1 in column w for each edge B-¿w: columns C and D. (Option B would be the row of the undirected version.)

My answer

Q6.

Quiz 5 V1 key (T251)

A directed graph has vertices 2, 3, 5, 7, 8, 9, 10, 11 and edges `5->11, 7->11, 7->8, 3->8, 3->10, 11->2, 11->9, 11->10, 8->9`. The adjacency list representation of the graph is an array of linked lists, where

(A) 4 such linked lists are nonempty.

(B) 5 such linked lists are nonempty.

(C) 6 such linked lists are nonempty.

(D) 8 such linked lists are nonempty.

Answer: (B). Only vertices with outgoing edges have nonempty lists: 3, 5, 7, 8, 11.

My answer

Q7. Quiz 5 V1 key (T251)

For the directed graph of the previous question (edges `5->11, 7->11, 7->8, 3->8, 3->10, 11->2, 11->9, 11->10, 8->9`), the number of vertices with out-degree 2 is equal to

(A) 1

(B) 2

(C) 3

(D) 4

Answer: (B). Vertices 7 (to 11, 8) and 3 (to 8, 10). Vertex 11 has out-degree 3; 5 and 8 have out-degree 1.

My answer

Q8. Quiz 5 Ver A key (T242)

Which of the following statements is **not** true regarding undirected graphs?

(A) The adjacency matrix representation is more efficient in edge insertion than adjacency list representation.

(B) The adjacency matrix representation is more efficient in an edge lookup than adjacency list representation.

(C) The adjacency matrix representation is more efficient in finding adjacent vertices than adjacency list representation.

(D) The adjacency matrix representation is more efficient in edge deletion than adjacency list representation.

Answer: (C). Insert, delete and lookup are $O(1)$ for the matrix versus $O(|V|)$ for lists; finding the neighbours of a vertex needs a full row scan ($O(|V|)$) in the matrix, which is never better than walking its linked list.

My answer

Q9. Old Exam Graphs

An undirected graph of n cities and m roads is stored as an **incidence matrix** (one row per city, one column per road, entry 1 if the road touches the city). In the worst case, what is the time complexity of finding whether two cities v_1 and v_2 are connected by a direct road?

(A) $O(1)$

(B) $O(n)$

(C) $O(m)$

(D) $O(n \cdot m)$

Answer: (C). We must scan the rows of v_1 and v_2 (m entries each) looking for a column in which both are 1.

My answer

Q10. Quiz 5 V1 key (T251)

A directed graph has edges `a->b, a->d, b->e, b->f, c->b, d->b, d->e, d->h, e->h, f->c, f->e, f->h, g->d, h->g, h->i, i->f`. Processing adjacent vertices in alphabetical order, the breadth-first traversal starting from vertex `a` is

(A) abdfehcgi

(B) abdfehcig

(C) abdefhcgi

(D) abdefhcig

Answer: (C). Queue: a; b, d; then from b: e, f; from d: h; from f: c; from h: g, i.

My answer

Q11. Quiz 5 V1 key (T251)

For the same graph (edges `a->b, a->d, b->e, b->f, c->b, d->b, d->e, d->h, e->h, f->c, f->e, f->h, g->d, h->g, h->i, i->f`), with adjacent vertices processed alphabetically, the **pre-order** depth-first traversal starting from vertex `a` is

(A) dgcfiheba

(B) dghcfieba

(C) abfcehgdi

(D) abehgdifc

Answer: (D). a → b → e → h → g → d (all its neighbours visited), back to h → i → f → c.

My answer

Q12. Quiz 5 V1 key (T251)

For the same graph, the **post-order** depth-first traversal starting from vertex `a` is

(A) dgcfiheba

(B) dghcfieba

(C) abfcehgdi

(D) abehgdifc

Answer: (A). Finishing order of the DFS of the previous question: d, g, c, f, i, h, e, b, a.

My answer

Q13. Quiz 5 V1 key (T251)

Using the general traversal algorithm, a post-order depth-first traversal of the directed graph with edges `a->b, a->e, d->a, d->e, d->f, e->f, c->b, c->f` starts from vertex `a`; when choosing from more than one vertex, alphabetical order is used. What is printed?

(A) f, e, d, b, a, c

(B) b, f, e, a, c, d

(C) b, e, f, a, c, d

(D) f, e, b, a, d, c

Answer: (B). From a: b finishes first, then e $\rightarrow$ f, so f, e, then a. The next unvisited vertex is c (its neighbours are visited), then d. (old key said f, e, d, b, a, c, which ignores the edge directions/alphabetical order.)

My answer

Q14. Quiz 5 Ver A key (T242)

An undirected graph has edges `1-3, 1-5, 1-8, 2-6, 3-6, 3-8, 4-5, 4-9, 4-10, 5-10, 6-7, 6-8, 6-10, 7-10`. Which is the correct post-order depth first traversal, starting from vertex 1 and traversing neighbours of a vertex in numerical order?

(A) 2, 9, 5, 4, 10, 7, 8, 6, 3, 1

(B) 2, 9, 4, 5, 10, 7, 6, 8, 3, 1

(C) 2, 5, 9, 4, 10, 7, 8, 6, 3, 1

(D) 1, 3, 6, 2, 7, 10, 4, 5, 9, 8

Answer: (C). Path 1, 3, 6, 2 (2 finishes), 6, 7, 10, 4, 5 (5 finishes), 4, 9 (9 finishes), then 4, 10, 7 finish, then 8, 6, 3, 1.

My answer

Q15. Quiz 5 key (T232)

Find the depth-first pre-order traversal of the directed graph G_A with edges `A->B, A->D, B->E, C->F, F->G`, starting from vertex A (general traversal; edges processed in alphabetical order of adjacent vertices).

(A) A B D E C F G

(B) A B E D C F G

(C) A D B E C F G

(D) A B E D C G F

Answer: (B). A, B, E, back to A, D; then the next unvisited vertex C, F, G.

My answer

Q16. Old Exam Graphs

A directed graph has edges `b->a, c->b, c->f, d->a, d->b, d->c, d->e, d->g, e->b, e->g, f->e, f->h, f->i, h->d, h->e, h->g, h->i`. The post order depth-first traversal of the graph, where vertices and adjacent vertices are processed in alphabetical order, is

(A) c,f,h,i,d,e,g,b,a

(B) a,b,e,i,d,g,h,f,c

(C) a,b,c,e,d,i,h,f,g

(D) a,b,g,e,d,i,h,f,c

Answer: (D). a and b finish immediately; from c: f $\rightarrow$ e $\rightarrow$ g (g, e finish), f $\rightarrow$ h $\rightarrow$ d (d finishes), h $\rightarrow$ i; then h, f, c finish.

My answer

Q17. Old Exam Graphs

For the same directed graph (edges `b->a, c->b, c->f, d->a, d->b, d->c, d->e, d->g, e->b, e->g, f->e, f->h, f->i, h->d, h->e, h->g, h->i`), the breadth-first traversal, where vertices and adjacent vertices are processed in alphabetical order, is

(A) a,b,c,f,e,h,i,g,d

(B) a,b,e,i,d,g,h,f,c

(C) a,b,g,e,d,i,h,f,c

(D) a,b,c,e,d,i,h,f,g

Answer: (A). a alone; b alone; from c: f; from f: e, h, i; from e: g; from h: d.

My answer

Q18. Activity 13-02 (T252)

An undirected graph has edges `A-B, A-S, S-C, S-G, C-D, C-E, C-F, E-H, F-G, G-H`. What is the post-order depth-first traversal starting from A (neighbours in alphabetical order)?

(A) B D F G H E C S A

(B) B D E F H G C S A

(C) A B S C D E H G F

(D) D B F G H E C S A

Answer: (A). Preorder is A B S C D E H G F; B and D finish first, then the chain E → H → G → F unwinds as F, G, H, E, followed by C, S, A.

My answer

Q19. Old Exam Graphs

A directed graph has edges `v1->v4, v1->v5, v1->v7, v2->v3, v2->v5, v2->v6, v3->v4, v3->v5, v4->v5, v5->v6, v5->v7, v6->v7`. Using the general traversal algorithm and visiting vertices in the order of their labels, the breadth-first traversal starting from vertex v1 is

(A) v1, v4, v5, v7, v6, v2, v3

(B) v1, v5, v4, v7, v6, v2, v3

(C) v1, v4, v5, v6, v7, v2, v3

(D) v1, v4, v5, v7, v2, v3, v6

Answer: (A). From v1: v4, v5, v7; from v5: v6. v2 and v3 are not reachable from v1, so a new traversal starts at v2, which enqueues v3.

My answer

Q20. Quiz 5 V1 key (T251)

Which of the following results from running the topological sort algorithm on the directed graph with edges `5->11, 7->11, 7->8, 3->8, 3->10, 11->2, 11->9, 11->10, 8->9`?

(A) 3 5 7 8 9 10 11 2

(B) 3 7 11 5 2 8 9 10

(C) 3 5 8 7 11 2 9 10

(D) 3 5 7 8 11 2 9 10

Answer: (D). It is the only list where every edge u-¿v has u before v (A puts 9 before 11, B puts 11 before 5, C puts 8 before 7).

My answer

Q21. Quiz 5 Ver A key (T242)

What is the necessary condition for a directed graph to have a topological sort solution?

(A) The graph must have at least one vertex with an in-degree of 0 in every iteration of the topological sort algorithm.

(B) The graph must have at least one vertex with an out-degree of 0 in every iteration of the topological sort algorithm.

(C) The graph must be weakly connected.

(D) The graph must be strongly connected.

Answer: (A). The algorithm repeatedly removes an in-degree-0 vertex; if at some step none exists, the remaining vertices contain a cycle and no topological order exists.

My answer

Q22. Old Exam Graphs

Consider the following statements with respect to the Topological Sort algorithm:
S1: The Topological Sort algorithm is used to arrange vertices in a directed acyclic graph, such that no vertex appears before its predecessor.
S2: The Topological Sort algorithm can be used to detect cycles in directed graphs.
S3: The Topological Sort algorithm can be used to generate minimum spanning trees in undirected graphs.
Then,

(A) Only statement S1 is correct.

(B) Only statement S2 is correct.

(C) Both statements S1 and S2 are correct.

(D) Both statements S2 and S3 are correct.

Answer: (C). If the topological order traversal cannot visit all vertices, the digraph has a cycle; it has nothing to do with spanning trees.

My answer

Q23. Quiz 5 key (T232)

Apply the topological sort algorithm on the directed graph G_B with edges `a->b, a->c, a->e, b->f, c->d, c->f, c->g, d->b, d->e, d->h, e->f, f->h, g->e, g->h`. When several vertices have in-degree 0, the alphabetically smallest is taken first. The resulting topological ordering is

(A) a b c d e f g h

(B) a c d g b e f h

(C) a c d b g e f h

(D) a c b d g e f h

Answer: (C). a; then c; then d; now b and g both have in-degree 0, take b, then g; then e, f, h. (B is a valid order but does not follow the alphabetical rule; A and D put b before d although d-¿b.)

My answer

Q24. Activity 13-02 (T252)

How many different topological orderings does the DAG with edges `A->B, A->C, A->D, B->C, B->D, C->E, D->E` have?

(A) 1

(B) 2

(C) 3

(D) 4

Answer: (B). A and B are forced first and E last; C and D can come in either order: A B C D E and A B D C E.

My answer

Q25. Old Exam Graphs

Consider the following statements with respect to shortest path problems on a graph $G = (V, E)$:
S1: Dijkstra's algorithm finds the shortest path between every pair of vertices in G.
S2: Dijkstra's algorithm may fail when edges are allowed to have negative weights.
S3: The shortest path problem is not well-defined when edges with negative weights are allowed.
Then,

(A) Only statement S1 is correct.

(B) Only statement S2 is correct.

(C) Both statements S1 and S2 are correct.

(D) Both statements S2 and S3 are correct.

Answer: (B). Dijkstra is single-source. With negative edges but no negative-cost cycle the problem is still well-defined; only a negative cycle makes it ill-defined.

My answer

Q26. Old Exam Graphs

A weighted directed graph has edges `s->u (6), s->x (7), u->v (5), v->u (-2), u->x (8), u->y (-4), x->v (-3), x->y (9), y->v (7), y->s (2)`. Which statement is correct?

(A) The shortest path problem is not well-defined because some weights are negative.

(B) The shortest path problem can be solved (there is no negative-cost cycle), but Dijkstra's algorithm is not applicable.

(C) Dijkstra's algorithm is applicable because the graph has no negative-cost cycle.

(D) The graph has a negative-cost cycle, so neither question has an answer.

Answer: (B). Every cycle has positive total cost (e.g. u,v,u = 3; u,y,v,u = 1), so shortest paths exist; but Dijkstra is guaranteed only for non-negative weights.

My answer

Q27.

Old Exam Graphs

Apply Dijkstra's algorithm on the weighted directed graph with edges `s->a (1)`, `s->b (5)`, `a->b (2)`, `a->c (2)`, `a->d (1)`, `b->d (2)`, `c->d (3)`, `c->e (1)`, `d->e (2)`, starting from vertex s. What are the final (distance, predecessor) of vertices `b` and `e`?

(A) b: (3, a), e: (4, d)

(B) b: (5, s), e: (4, c)

(C) b: (3, a), e: (4, c)

(D) b: (3, a), e: (5, d)

Answer: (A). Order made known: s(0), a(1), d(2), b(3 via a), c(3 via a), e(4 via d); c's edge gives e = 3+1 = 4, which is not smaller, so e keeps predecessor d.

My answer

Q28.

Old Exam Graphs

A weighted directed graph has edges `1->4 (13)`, `1->7 (2)`, `2->1 (1)`, `3->1 (25)`, `3->2 (2)`, `3->5 (30)`, `5->2 (5)`, `5->6 (4)`, `5->8 (14)`, `6->3 (11)`, `6->9 (9)`, `7->4 (12)`, `7->6 (17)`, `7->10 (8)`, `8->9 (3)`, `8->11 (6)`, `9->5 (15)`, `10->9 (8)`, `11->10 (7)`. Dijkstra's algorithm is run from vertex 8. Using the predecessor column, the shortest path from 8 to 1 and its cost are

(A) 8, 9, 5, 2, 1 with cost 24

(B) 8, 5, 2, 1 with cost 20

(C) 8, 11, 10, 9, 5, 2, 1 with cost 42

(D) 8, 9, 5, 6, 3, 1 with cost 58

Answer: (A). pred(1) = 2, pred(2) = 5, pred(5) = 9, pred(9) = 8; cost 3+15+5+1 = 24. (There is no edge 8-¿5; the edge is 5-¿8.)

My answer

Q29.

Old Exam Graphs

Run Dijkstra's algorithm from vertex A on the directed weighted graph with edges `A->B (8)`, `A->C (14)`, `A->G (5)`, `B->D (4)`, `C->D (5)`, `D->F (1)`, `E->C (2)`, `E->F (9)`, `F->H (6)`, `G->A (1)`, `G->E (6)`, `H->B (3)`, `H->G (3)`. What are the final shortest distances (and predecessors) of C and H?

(A) C: 14 (A), H: 19 (F)

(B) C: 13 (E), H: 19 (F)

(C) C: 13 (E), H: 20 (F)

(D) C: 13 (E), H: 22 (E)

Answer: (B). Known order A(0), G(5), B(8), E(11), D(12), C(13 via E), F(13 via D), H(19 via F).

My answer

Q30.

Quiz 5 V1 key (T251)

A directed graph has 6 vertices and 8 edges: `A->B (4), A->C (2), C->B (7), C->D (6), B->D (1), D->F (3), D->E (5), F->E (1)`. Running Dijkstra's algorithm from vertex A, in which order do the vertices become known (active vertex of each pass)?

(A) A, C, B, D, F, E

(B) A, C, B, D, E, F

(C) A, B, C, D, F, E

(D) A, C, D, B, F, E

Answer: (A). Distances: C 2, B 4, D 5 (via B), F 8 (via D), E 9 (via F, better than 10 via D).

My answer

Q31.

Activity 13-03 (T252)

Perform Dijkstra's algorithm from A on the undirected weighted graph with edges `A-B (4), A-H (8), B-C (8), B-H (11), C-D (7), C-F (4), C-I (2), D-E (9), D-F (14), E-F (10), F-G (2), G-H (1), G-I (6), H-I (7)`. What are the final shortest distances (and predecessors) of E and I?

(A) E: 21 (F), I: 14 (C)

(B) E: 21 (D), I: 15 (H)

(C) E: 22 (F), I: 14 (C)

(D) E: 19 (F), I: 15 (G)

Answer: (A). Final: B 4 (A), H 8 (A), G 9 (H), F 11 (G), C 12 (B), I 14 (C), D 19 (C), E 21 (F; via D it would be 28).

My answer

Q32.

Old Exam Graphs

Apply Kruskal's algorithm to the weighted undirected graph with edges `V1-V7 (1), V3-V4 (3), V2-V7 (4), V3-V7 (9), V2-V3 (15), V4-V7 (16), V4-V5 (17), V1-V2 (20), V1-V6 (23), V5-V7 (25), V5-V6 (28), V6-V7 (36)`. What is the cost of the minimum spanning tree, and which edge is inserted last?

(A) 57, V1-V6

(B) 57, V4-V5

(C) 53, V1-V6

(D) 61, V2-V3

Answer: (A). Inserted: V1-V7 (1), V3-V4 (3), V2-V7 (4), V3-V7 (9), V4-V5 (17), V1-V6 (23); 15, 16 and 20 are rejected (cycles). Total 57.

My answer

Q33.

Activity 13-04 (T252)

Consider the undirected graph with edges `a-b (1), a-c (7), b-c (5), b-d (3), b-e (4), c-d (6), d-e (2)`. If we apply Prim's algorithm (starting at a) and Kruskal's algorithm independently, which algorithm selects the edge with weight 2 before it is selected by the other algorithm?

(A) Prim's algorithm

(B) Kruskal's algorithm

(C) Both select it at the same step

(D) Neither algorithm selects it

Answer: (B). Kruskal: a-b (1), d-e (2) is the 2nd edge. Prim from a: a-b (1), b-d (3), d-e (2) is the 3rd edge.

My answer

Q34.

Activity 13-04 (T252)

An undirected graph has edges `a-b (5), a-c (3), b-c (4), b-d (6), b-e (2), c-d (5), c-f (6), d-e (6), d-f (6), e-f (3), e-g (5), f-g (4)`. Which one of the following is **not** the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?

(A) (b,e), (e,f), (a,c), (b,c), (f,g), (c,d)

(B) (b,e), (e,f), (a,c), (f,g), (b,c), (c,d)

(C) (b,e), (a,c), (e,f), (b,c), (f,g), (c,d)

(D) (b,e), (e,f), (b,c), (a,c), (f,g), (c,d)

Answer: (D). Kruskal takes edges in non-decreasing weight; (b,c) of weight 4 cannot be added before (a,c) of weight 3. Equal-weight edges may come in either order.

My answer

Q35.

Activity 13-04 (T252)

An undirected graph has edges `A-B (2), A-C (1), A-D (4), B-C (3), C-D (2), C-E (9), C-F`

`(8), C-G (4), D-F (6), F-G (1)`. If we start Prim's algorithm at vertex C, which of the following edges will **not** be selected by the algorithm?

(A) (C,A)

(B) (C,E)

(C) (A,B)

(D) (C,B)

Answer: (D). Prim adds C-A (1), A-B (2), C-D (2), C-G (4), G-F (1), C-E (9); B is reached by A-B (2), so C-B (3) is never chosen.

My answer

Q36.

Old Exam Graphs

Apply Prim's algorithm starting from A to the weighted undirected graph with edges `A-B (22), A-C (9), A-D (12), B-C (35), B-F (36), B-H (34), C-D (4), C-E (65), C-F (42), D-E (33), D-I (30), E-F (18), E-G (23), F-G (39), F-H (24), G-H (25), G-I (21), H-I (19).` In which order are the vertices added to the tree?

(A) A, C, D, B, I, H, G, E, F

(B) A, C, D, I, H, G, E, F, B

(C) A, C, D, B, E, F, G, H, I

(D) A, C, B, D, I, H, G, E, F

Answer: (A). Edges chosen: A-C 9, C-D 4, A-B 22, D-I 30, I-H 19, I-G 21, G-E 23, E-F 18; total cost 146.

My answer

Q37.

Old Exam Graphs

For the directed graph with edges `b->a, c->b, c->f, d->a, d->b, d->c, d->e, d->g, e->b, e->g, f->e, f->h, f->i, h->d, h->e, h->g, h->i`, the number of strongly connected components is equal to

(A) 8

(B) 7

(C) 6

(D) 5

Answer: (C). c-¿f-¿h-¿d-¿c is a cycle, so {c, d, f, h} is one SCC; a, b, e, g, i are each alone (no path back): 6 SCCs.

My answer

Q38.

Old Exam Graphs

Consider the directed graph with edges `a->d, b->a, b->e, b->f, c->b, d->b, d->e, e->f, f->c, f->i, g->d, g->h, h->d, h->e, h->f, i->h`. The number of strongly connected components is equal to

(A) 4

(B) 3

(C) 2

(D) 1

Answer: (C). {g} (no edge enters g) and {a, b, c, d, e, f, h, i}: e.g. a-¿d-¿b-¿a, b-¿f-¿c-¿b, f-¿i-¿h-¿d, h-¿e-¿f.

My answer

Q39.

Old Exam Graphs

A directed graph has 8 vertices and edges `1->2, 2->3, 3->4, 4->1, 3->5, 5->6, 6->7, 7->5, 6->8, 7->8`. The number of strongly connected components is equal to

(A) 5

(B) 4

(C) 3

(D) 2

Answer: (C). {1, 2, 3, 4} (cycle), {5, 6, 7} (cycle), and {8} (no outgoing edges).

My answer

Q40.

Activity 13-05 (T252)

Run the SCC (Kosaraju) algorithm on the directed graph with vertices 0–8 and edges `0->1, 1->2, 2->3, 3->0, 2->4, 4->5, 5->6, 6->4, 7->6, 7->8`. The strongly connected components are

(A) {0, 1, 2, 3}, {4, 5, 6}, {7}, {8}

(B) {0, 1, 2, 3, 4, 5, 6}, {7, 8}

(C) {0, 1, 2, 3}, {4, 5, 6, 7}, {8}

(D) {0, 1, 2, 3}, {4, 5, 6}, {7, 8}

Answer: (A). 0-1-2-3 and 4-5-6 are cycles; nothing returns from 4, 5, 6 to 2, and 7 and 8 lie on no cycle: 4 SCCs.

My answer

Part B: Programming

The unweighted questions use the slides' class (vertices are numbered $0..\texttt{numVertices}-1$; for an undirected graph every edge is stored in both lists):

```
import java.util.*;
public class GraphAsAdjacencyList {
    int numVertices;
    LinkedList<Integer> list[];   // list[u] = vertices adjacent to u
    public GraphAsAdjacencyList(int numVertices) {
        this.numVertices = numVertices;
        list = new LinkedList[numVertices];
        for (int i = 0; i < numVertices; i++) list[i] = new LinkedList<>();
    }
}
```

Q41. [new]

Add the following methods to the directed `GraphAsAdjacencyList`: `int outDegree(int v)`, `int inDegree(int v)` (the number of edges incident to `v`), and `int countSinks()` that returns the number of vertices with out-degree 0. State the complexity of `inDegree`.

Answer:

```
public int outDegree(int v) { return list[v].size(); }

public int inDegree(int v) {            // O(|V| + |E|): traverse all lists
    int count = 0;
    for (int u = 0; u < numVertices; u++)
        for (int w : list[u])
            if (w == v) count++;
    return count;
}

public int countSinks() {
    int c = 0;
    for (int v = 0; v < numVertices; v++)
        if (outDegree(v) == 0) c++;
    return c;
}
```

My answer

Q42. [new]

Write `GraphAsAdjacencyList transpose()` that returns a new directed graph with every edge reversed, and `boolean isStronglyConnected()` that returns true if every vertex can reach every other

vertex. Use the idea: the graph is strongly connected iff all vertices are reachable from vertex 0 in the graph **and** in its transpose.

Answer:

```
public GraphAsAdjacencyList transpose() {
    GraphAsAdjacencyList t = new GraphAsAdjacencyList(numVertices);
    for (int u = 0; u < numVertices; u++)
        for (int v : list[u])
            t.list[v].add(u);          // edge u->v becomes v->u
    return t;
}

private int countReachable(int s) {    // iterative DFS
    boolean[] visited = new boolean[numVertices];
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(s); visited[s] = true;
    int count = 0;
    while (!stack.isEmpty()) {
        int u = stack.pop(); count++;
        for (int w : list[u])
            if (!visited[w]) { visited[w] = true; stack.push(w); }
    }
    return count;
}

public boolean isStronglyConnected() {
    if (numVertices == 0) return true;
    return countReachable(0) == numVertices
        && transpose().countReachable(0) == numVertices;
}
```

My answer

Q43. [new]

Write `List<Integer> breadthFirst(int start)` that returns the vertices reachable from `start` in breadth-first order. Use a queue and a boolean array so that a vertex is never enqueued more than once (neighbours are processed in the order they appear in the adjacency list).

Answer:

```
public List<Integer> breadthFirst(int start) {
    List<Integer> order = new ArrayList<>();
    boolean[] visited = new boolean[numVertices];
    Queue<Integer> queue = new LinkedList<>();
```

```
    queue.add(start); visited[start] = true;      // mark when enqueued
    while (!queue.isEmpty()) {
        int v = queue.remove();
        order.add(v);                              // visit v
        for (int w : list[v])
            if (!visited[w]) { visited[w] = true; queue.add(w); }
    }
    return order;
}
```

My answer

Q44. [new]

Write `void depthFirst(int start)` that prints the preorder and the postorder depth-first traversals of the whole graph, using the general traversal algorithm: start from `start`, then restart from every vertex that is still unvisited (in increasing order). Use a recursive helper.

Answer:

```
private boolean[] visited;
private List<Integer> pre, post;

public void depthFirst(int start) {
    visited = new boolean[numVertices];
    pre = new ArrayList<>(); post = new ArrayList<>();
    dfs(start);
    for (int v = 0; v < numVertices; v++)
        if (!visited[v]) dfs(v);
    System.out.println("Preorder:  " + pre);
    System.out.println("Postorder: " + post);
}

private void dfs(int u) {
    visited[u] = true;
    pre.add(u);                 // visit on entry  -> preorder
    for (int w : list[u])
        if (!visited[w]) dfs(w);
    post.add(u);                // visit on exit   -> postorder
}
```

My answer

Q45. [new]

Write `List<Integer> topologicalSort()` for a directed graph using the in-degree algorithm of the slides: compute all in-degrees, repeatedly remove a vertex of in-degree 0 and update the in-degrees of its successors. Throw an `IllegalStateException` if the graph has a cycle (not all vertices can be visited).

Answer:

```
public List<Integer> topologicalSort() {
    int[] inDeg = new int[numVertices];
    for (int u = 0; u < numVertices; u++)
        for (int v : list[u]) inDeg[v]++;
    Queue<Integer> queue = new LinkedList<>();
    for (int v = 0; v < numVertices; v++)
        if (inDeg[v] == 0) queue.add(v);
    List<Integer> order = new ArrayList<>();
    while (!queue.isEmpty()) {
        int v = queue.remove();
        order.add(v);                       // visit v
        for (int w : list[v]) {             // delete v's emanating edges
            inDeg[w]--;
            if (inDeg[w] == 0) queue.add(w);
        }
    }
    if (order.size() != numVertices)
        throw new IllegalStateException("No topological sort: graph has a cycle");
    return order;
}
```

My answer

Q46. [new]

A weighted digraph is stored as `List<List<Edge>> graph`, where `graph.get(u)` holds the edges leaving `u`:

```
class Edge { int to, weight; Edge(int to, int weight) { this.to = to; this.weight = weight;
    } }
class Node { int node, dist; Node(int node, int dist) { this.node = node; this.dist = dist;
    } }
```

Write `static int[] dijkstra(List<List<Edge>> graph, int start, int[] parent)` that returns the shortest distances from `start` (non-negative weights) and fills `parent` with each vertex's predecessor (`-1` if none), using a `PriorityQueue<Node>` as a min-heap. Also write `static String path(int[] parent, int v)` that returns the path from the source to `v`, e.g. `"0-1-3"`.

Answer:

```
static int[] dijkstra(List<List<Edge>> graph, int start, int[] parent) {
    int n = graph.size();
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    Arrays.fill(parent, -1);
    dist[start] = 0;
    PriorityQueue<Node> pq = new PriorityQueue<>(Comparator.comparingInt(x -> x.dist));
    pq.add(new Node(start, 0));
    boolean[] visited = new boolean[n];
    while (!pq.isEmpty()) {
        int u = pq.poll().node;
        if (visited[u]) continue;           // stale heap entry
        visited[u] = true;                  // u is now known
        for (Edge e : graph.get(u)) {
            int v = e.to, w = e.weight;
            if (!visited[v] && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                parent[v] = u;
                pq.add(new Node(v, dist[v]));
            }
        }
    }
    return dist;
}

static String path(int[] parent, int v) {
    if (parent[v] == -1) return "" + v;
```

```
    return path(parent, parent[v]) + "-" + v;
}
```

My answer

Q47. [new]

An undirected, connected, weighted graph is stored as an adjacency matrix `int[][] w` where `w[u][v] == 0` means there is no edge. Write `static int primMST(int[][] w, int s, int[] pred)` that runs Prim's algorithm from `s`, fills `pred` with the tree parent of each vertex (`-1` for `s`) and returns the total cost of the minimum spanning tree. Use the $O(|V|^2)$ version: in each step pick the vertex not in the tree with the smallest `dist`.

Answer:

```
static int primMST(int[][] w, int s, int[] pred) {
    int n = w.length;
    int[] dist = new int[n];              // cheapest edge connecting v to the tree
    boolean[] inT = new boolean[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    Arrays.fill(pred, -1);
    dist[s] = 0;
    int total = 0;
    for (int k = 0; k < n; k++) {
        int v = -1;
        for (int i = 0; i < n; i++)
            if (!inT[i] && (v == -1 || dist[i] < dist[v])) v = i;
        if (dist[v] == Integer.MAX_VALUE)
            throw new IllegalStateException("Graph is not connected");
        inT[v] = true;
        total += dist[v];
        for (int u = 0; u < n; u++)       // d(u) = min(w(v,u), d(u))
            if (!inT[u] && w[v][u] != 0 && w[v][u] < dist[u]) {
                dist[u] = w[v][u];
                pred[u] = v;
            }
    }
    return total;
}
```

My answer

Q48. [new]

Edges of an undirected weighted graph with vertices $0..n-1$ are given as a list of

```
class WEdge { int u, v, weight; WEdge(int u, int v, int w) { this.u = u; this.v = v; this.
    weight = w; } }
```

Write `static List<WEdge> kruskal(int numVertices, List<WEdge> edges)` that returns the edges of a minimum spanning tree. Sort the edges by weight; to test whether an edge creates a cycle, keep a component label `comp[i]` for every vertex (an edge is accepted only if its endpoints have different labels; then relabel one component).

Answer:

```
static List<WEdge> kruskal(int numVertices, List<WEdge> edges) {
    List<WEdge> sorted = new ArrayList<>(edges);
    sorted.sort(Comparator.comparingInt(e -> e.weight));
    int[] comp = new int[numVertices];
    for (int i = 0; i < numVertices; i++) comp[i] = i;
    List<WEdge> T = new ArrayList<>();
    for (WEdge e : sorted) {
        if (T.size() == numVertices - 1) break;   // |V|-1 edges: done
        int cu = comp[e.u], cv = comp[e.v];
        if (cu != cv) {                           // no cycle
            T.add(e);
            for (int i = 0; i < numVertices; i++) // merge the two trees
                if (comp[i] == cv) comp[i] = cu;
        }
    }
    return T;
}
```

My answer

Q49. [new]

For an undirected `GraphAsAdjacencyList`, write `int[] connectedComponents()` that labels the vertices so that all vertices in the same connected component get the same label $0, 1, 2, \ldots$ and vertices in different components get different labels. Then write `boolean isConnected()` using it.

Answer:

```
public int[] connectedComponents() {
    int[] label = new int[numVertices];
    Arrays.fill(label, -1);                    // -1 = not visited
    int componentCount = 0;
    for (int v = 0; v < numVertices; v++)
        if (label[v] == -1) {
            labelDFS(v, componentCount, label);
            componentCount++;
        }
    return label;
}

private void labelDFS(int u, int c, int[] label) {
    label[u] = c;
    for (int w : list[u])
        if (label[w] == -1) labelDFS(w, c, label);
}

public boolean isConnected() {
    for (int x : connectedComponents())
        if (x != 0) return false;
    return true;
}
```

My answer

Q50. [new]

Write `boolean hasCycle()` for an undirected (simple) `GraphAsAdjacencyList` using the DFS method of the slides: keep `startTime[v]` (0 = unvisited) and `pred[v]`; a cycle exists iff DFS finds an already-visited neighbour `u` of `v` that is not `pred[v]`. Handle disconnected graphs.

Answer:

```
private int[] startTime, pred;
private int time;

public boolean hasCycle() {
    startTime = new int[numVertices];
    pred = new int[numVertices];
    Arrays.fill(pred, -1);
    time = 1;
    for (int v = 0; v < numVertices; v++)
        if (startTime[v] == 0 && cycleDetectionDFS(v)) return true;
    return false;
}

private boolean cycleDetectionDFS(int v) {
    startTime[v] = time++;
    for (int u : list[v]) {
        if (startTime[u] == 0) {
            pred[u] = v;
            if (cycleDetectionDFS(u)) return true;
        } else if (u != pred[v]) {
            return true;                    // visited and not the parent
        }
    }
    return false;
}
```

My answer